

Customer Retention Engine

Nadir Askarov

Valiyyaddin Aliyev

Javidan Muradzada

Matin Khalilov

April 17, 2026

Contents

1	Problem Overview	2
2	Data Description and Cleaning	2
3	Exploratory Data Analysis	3
4	Feature Engineering and Preprocessing	5
5	Model Building	6
6	Results and Model Comparison	6
7	Best Model Selection	7
8	Model Interpretation	8
9	Recommendation System	9
10	Conclusion	11

1 Problem Overview

This report summarizes the experiment in `notebooks/experiment.ipynb`, where the goal is to predict customer churn ($\text{Churn} = 0/1$). The business problem solved by this project is retention management system. Identifying customers likely to churn early and recommending interventions to reduce churn, protect recurring revenue, and prioritize retention operations `notebooks/recommend.ipynb`. Different classification models were trained and compared, then the best model was selected using F1 score.

2 Data Description and Cleaning

The dataset was loaded from:

```
../data/customer_churn_dataset-training-master.csv
```

Data sample

	CustomerID	Age	Gender	Tenure	Usage Frequency	Support Calls	Payment Delay	Subscription Type	Contract Length	Total Spend	Last Interaction	Churn
0	2.0	30.0	Female	39.0	14.0	5.0	18.0	Standard	Annual	932.0	17.0	1.0
1	3.0	65.0	Female	49.0	1.0	10.0	8.0	Basic	Monthly	557.0	6.0	1.0
2	4.0	55.0	Female	14.0	4.0	6.0	18.0	Basic	Quarterly	185.0	3.0	1.0
3	5.0	58.0	Male	38.0	21.0	7.0	7.0	Standard	Monthly	396.0	29.0	1.0
4	6.0	23.0	Male	32.0	20.0	5.0	8.0	Basic	Monthly	617.0	20.0	1.0

Figure 1: Sample rows from the customer churn dataset

Initial columns (12): CustomerID, Age, Gender, Tenure, Usage Frequency, Support Calls, Payment Delay, Subscription Type, Contract Length, Total Spend, Last Interaction, Churn.

Column descriptions

Column	Description
CustomerID	Unique identifier for each customer record. Used only for identification.
Age	Customer age in years. Used for exploration and transformed into AgeGroup for modeling.
Gender	Customer gender category (e.g., Male/Female). Categorical input feature.
Tenure	Duration of customer relationship with the service. Higher/lower values may indicate loyalty differences.
Usage Frequency	How frequently the customer uses the service/product. Behavioral engagement signal.
Support Calls	Number of customer support interactions. Higher values can indicate dissatisfaction or service issues.
Payment Delay	Delay in customer payments. Higher delay can indicate financial or engagement risk.
Subscription Type	Plan type selected by the customer (e.g., Basic, Standard, Premium).
Contract Length	Billing/contract cycle category (e.g., Monthly, Quarterly, Annual).
Total Spend	Total spending amount by customer.
Last Interaction	Latest interaction activity .
Churn	Target label: 1 = churned customer, 0 = retained customer.

Table 1: Definitions of all columns in the dataset

Data quality checks found one row with missing values across all columns. After dropping missing rows:

```
df.shape = (440832, 12)
```

Then **CustomerID** was removed because it is an identifier and not a predictive behavioral feature.

3 Exploratory Data Analysis

Category distributions

Key distributions from notebook outputs:

- **AgeGroup**: Senior 156,893; Adult 117,210; Youth 102,693; Child 64,036
- **Gender**: Male 250,252; Female 190,580
- **Subscription Type**: Standard 149,128; Premium 148,678; Basic 143,026
- **Contract Length**: Annual 177,198; Quarterly 176,530; Monthly 87,104

Churn-wise averages

Feature	Not Churn (0)	Churn (1)
Tenure	32.2818	30.4736
Usage Frequency	16.2606	15.4617
Age	36.2630	41.7473
Payment Delay	10.0155	15.2177
Total Spend	749.9531	541.2855
Support Calls	1.5864	5.1449

Table 2: Average feature values by churn class

Main pattern: churned customers are older on average, have higher payment delay, much more support calls, lower total spend, and slightly lower tenure/usage.

EDA figures

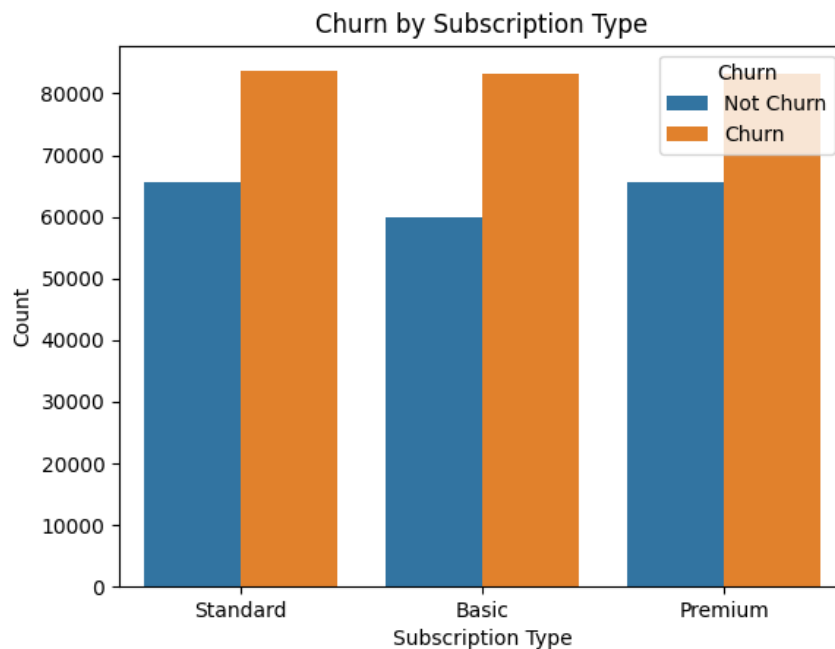


Figure 2: Churn counts by subscription type

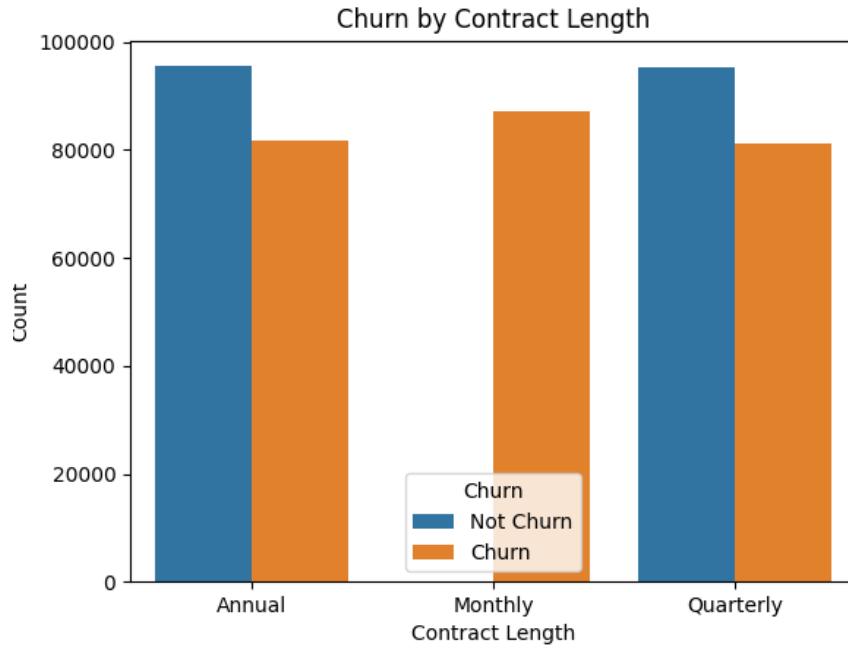


Figure 3: Churn counts by contract length

4 Feature Engineering and Preprocessing

Implemented feature engineering:

- Created `AgeGroup` from `Age` using bins $[15, 25)$, $[25, 35)$, $[35, 45)$, $[45, 66)$ with labels `Child`, `Youth`, `Adult`, `Senior`.
- Dropped raw `Age` from model input after deriving `AgeGroup`.

Model input/target:

- `X = df.drop(['Churn', 'Age'])`
- `y = df['Churn'].astype(int)`

Preprocessing pipeline:

- `StandardScaler` for numerical features
- `OneHotEncoder(handle_unknown='ignore')` for categorical features: `Gender`, `Subscription Type`, `Contract Length`, `AgeGroup`

Train/test split:

- 80/20 split with `random_state=42`, `stratify=y`
- Test support from reports: 88,167 rows (38,167 non-churn, 50,000 churn)

5 Model Building

To ensure a fair comparison, all models were trained on the same stratified 80/20 split and evaluated on the same test set after identical preprocessing (numerical scaling + categorical one-hot encoding). This design isolates model behavior as the main source of performance differences.

The notebook trained and evaluated four models:

1. Logistic Regression (`max_iter=1000`)
2. Decision Tree (`random_state=42`)
3. Random Forest (`n_estimators=100, random_state=42`)
4. Gradient Boosting (`n_estimators=100, random_state=42`)

Each model contributes a different learning bias:

- **Logistic Regression:** linear baseline; useful to check whether churn can be separated with a simple linear boundary.
- **Decision Tree:** captures non-linear thresholds and feature interactions, but can be unstable as a single-tree learner.
- **Random Forest:** bagging ensemble of trees; reduces variance and usually improves generalization on tabular data.
- **Gradient Boosting:** sequentially improves weak learners by focusing on residual errors; often strong on structured datasets.

Metrics used for comparison: Accuracy and F1

The primary selection criterion is **F1 score**, because churn prediction requires balancing missed churners (false negatives) and unnecessary interventions (false positives), not only maximizing overall accuracy.

6 Results and Model Comparison

Model	Accuracy	F1 Score
Random Forest	0.979221	0.981407
Gradient Boosting	0.977509	0.979871
Decision Tree	0.964080	0.968420
Logistic Regression	0.897660	0.907845

Table 3: Performance comparison from notebook output

Ranking by F1 is: **Random Forest** > **Gradient Boosting** > **Decision Tree** > **Logistic Regression**.

Detailed comparison:

- **Random Forest** achieved the best overall performance (Accuracy 0.9792, F1 0.9814).
- **Gradient Boosting** was very close (F1 0.9799), only about 0.0015 below Random Forest.

- **Decision Tree** performed clearly lower than both ensembles (F1 0.9684), showing weaker generalization than ensemble methods.
- **Logistic Regression** had the lowest scores (F1 0.9078), indicating that a linear decision boundary is insufficient for this churn pattern.

Class-wise reports and confusion matrices show that ensemble models (Random Forest, Gradient Boosting) reduce both false positives and false negatives compared to single-tree and linear baselines.

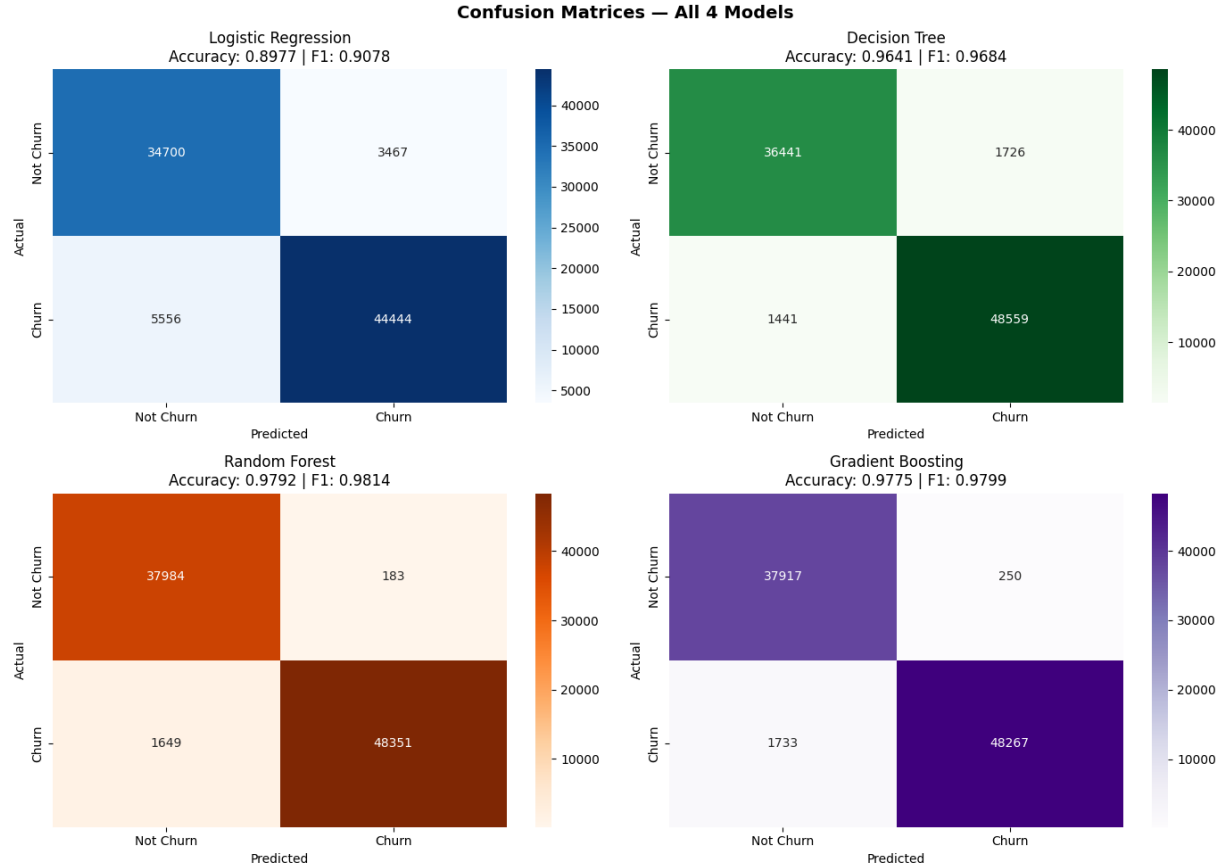


Figure 4: Confusion matrices for all four models

7 Best Model Selection

The final model choice followed a metric-first policy:

1. Maximize **F1 score** (primary business metric for churn intervention quality).
2. Use **Accuracy** as a secondary consistency check.
3. Confirm balanced class-wise behavior from the classification reports and confusion matrix.

Based on this policy, the notebook selected **Random Forest**:

- Accuracy: 0.9792

- F1 Score: 0.9814

Although Gradient Boosting is highly competitive, Random Forest remains the top model on both reported metrics and therefore becomes the production candidate. The saved artifact (`best_model.joblib`) is then used by the recommendation engine to generate customer-level retention actions.

8 Model Interpretation

Permutation importance was computed for the selected model.

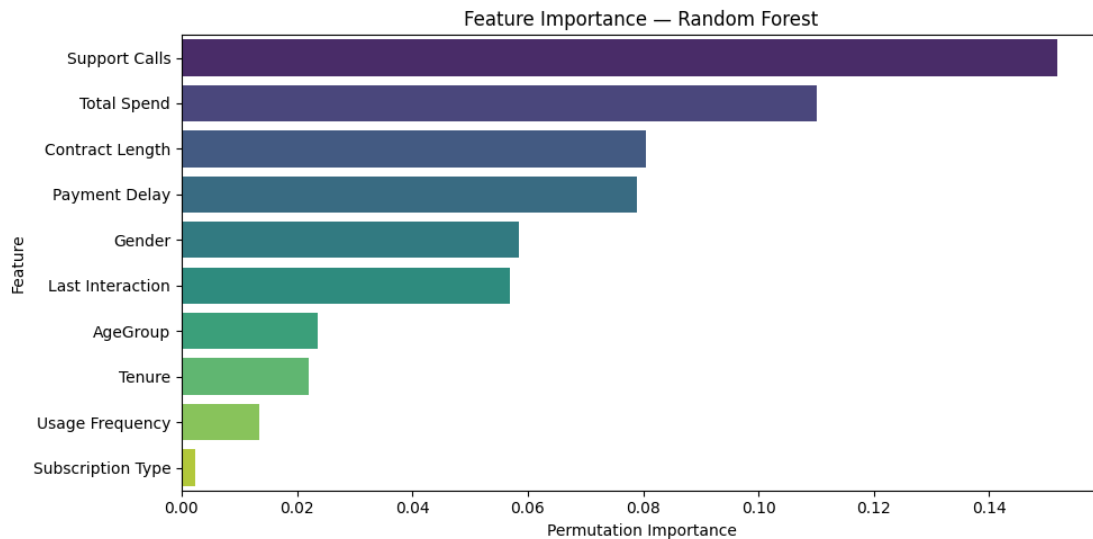


Figure 5: Permutation feature importance for Random Forest

Top drivers were:

1. Support Calls (0.1518)
2. Total Spend (0.1102)
3. Contract Length (0.0804)
4. Payment Delay (0.0789)

The notebook saved trained artifacts in `../models/`, including: `best_model.joblib` (Random Forest), all individual models, preprocessing pipeline, and column metadata.

A custom inference example in the notebook used the best model and returned:

- Predicted Churn: 0 (Not Churn)
- Churn Probability: 0.00%
- Retention Probability: 100.00%

9 Recommendation System

After model training, the project adds a **churn-prevention recommendation engine**. Instead of recommending products, this module recommends **minimal actionable customer-level interventions** that can flip a predicted outcome from `Churn` to `Not Churn`.

Purpose

For each customer profile:

1. Predict churn class and churn probability using the saved best model (Random Forest).
2. If customer is low risk (`Not Churn`), return no action.
3. If customer is high risk (`Churn`), search for the smallest feasible feature changes that reverse the prediction.

Loaded artifacts and inputs

The notebook loads:

- `../models/best_model.joblib`
- `../models/preprocessing_pipeline.joblib`
- `../models/column_info.joblib`

Model input columns used by this module: `Gender`, `Tenure`, `Usage Frequency`, `Support Calls`, `Payment Delay`, `Subscription Type`, `Contract Length`, `Total Spend`, `Last Interaction`, `AgeGroup`.

Actionable retention levers

Feature	Direction	Step	Business interpretation
Total Spend	Increase	+50	Offer discount/promo or bundled value to raise spend commitment
Payment Delay	Decrease	-1 day	Payment reminders, auto-pay enrollment, billing assistance
Support Calls	Decrease	-1 call	Proactive issue resolution to reduce escalations
Contract Length	Change category	Monthly/Quarterly/Annual	Move to a more stable contract option when beneficial

Table 4: Actionable features and intervention logic in `recommend.ipynb`

Algorithm design

The recommendation logic is a **counterfactual search with greedy minimal changes**:

1. Compute baseline prediction and churn probability.
2. Try each actionable feature independently.

3. For numeric features, use binary search to find the smallest change that flips prediction.
4. For contract type, test alternative categories (upgrade path first, then downgrade).
5. If one feature can flip prediction, pick the lowest-effort single change.
6. If not, test combinations of 2, 3, ... features and minimize each changed feature while keeping the flipped outcome.
7. If no feasible intervention flips prediction, return high-risk escalation.

Returned recommendation states

The engine returns one of four statuses:

- **NO_ACTION**: customer already predicted as non-churn.
- **SINGLE_CHANGE**: one minimal intervention is enough.
- **MULTI_CHANGE**: multiple coordinated interventions are needed.
- **HIGH_RISK**: model cannot be flipped with allowed actions; escalate to retention team.

Worked example from notebook output

Input customer profile in the notebook:

```
Female, Tenure=27, Usage Frequency=6, Support Calls=15, Payment Delay=29,
Subscription Type=Premium, Contract Length=Quarterly, Total Spend=233,
Last Interaction=14, AgeGroup=Adult
```

Observed output:

- Initial prediction: **Churn** with probability **100.00%**
- Recommended status: **MULTI_CHANGE**
- Coordinated interventions:
 - **Total Spend**: 233 → 533
Business action: Offer discount/promo or bundled value to increase customer commitment and perceived value.
 - **Payment Delay**: 29 → 20
Business action: Apply payment reminders, enable auto-pay, or provide billing assistance to reduce delays.
 - **Support Calls**: 15 → 4
Business action: Proactively resolve issues and improve service quality to reduce repeated support interactions.
- New churn probability after intervention: **16.00%**

This demonstrates the practical objective of the module: convert model predictions into concrete retention actions with minimal operational effort.

10 Conclusion

The pipeline is complete from cleaning and EDA through feature engineering, model comparison, and interpretation. Random Forest achieved the best F1 score and is the final selected model for churn prediction in this project. And recommendation system is used to prevent customer from leaving company

References

1. L. Breiman, “Random Forests,” *Machine Learning*, 45(1), 5–32, 2001.
2. F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, 12, 2825–2830, 2011. <https://scikit-learn.org>
3. M. Shahid Azeem, “Customer Churn Dataset,” Kaggle. <https://www.kaggle.com/datasets/muhammadshahidazeem/customer-churn-dataset>